

Univ.-Prof. Dr. Daniel Große
DI Simon Reitinger & Katharina Ruep, M.Sc.

21. Mai 2026

Übung 6 zur Vorlesung Rechnerarchitektur

Abgabe bis **Donnerstag 28 Mai, 2026 10:15** via EPICS: <https://ep.ics.jku.at>.

Aufgabe 1 – Multicycle States

(2+2 Punkte)

Öffne SonicRV¹ und verwende den Prozessor `ics-edu-rv32i-mc`.

- Schreibe ein Assembly Program, sodass alle Zustände des Multicycle Prozessors mindestens einmal besucht werden. Füge den Code in dein Abgabe-PDF ein.
- In welchen Zuständen wird die ALU verwendet? Beschreibe, welche Berechnung die ALU in diesen Zuständen durchführt (Ein Satz pro Zustand ist ausreichend).

Aufgabe 2 – Multicycle RISC-V Entwicklung

(3+7 Punkte)

Ändere den in der Vorlesung vorgestellten Multicycle RISC-V Prozessor folgendermaßen.

- Ein weiterer Instruktionstyp, der in unserem Multicycle RISC-V Modell zur Vollständigkeit noch fehlt ist der **J-Typ**. Um diesen Instruktionstypen zu unterstützen, erweitere soweit nötig den Schaltplan in Abb. 2, die FSM in Abb. 1, und die ImmSrc Encoding Tabelle 1 des Prozessors so, dass die Instruktion JAL ausgeführt werden kann. Zeichne darüber hinaus für **jeden State** der JAL-Instruktion den dazugehörigen Datenpfad im Schaltplan in Abb. 2 ein inkl. Markierungen bzw. unterschiedliche Farben je State.
- Das bestehende Register File wird durch ein Register File mit nur einem einzigen Adress- bzw. Daten-Port ersetzt (siehe Abb. 3). Das neue Register File legt auf RD das Datum von Adresse A an. Falls WE auf 1 gesetzt ist, wird das Datum von WD auf Adresse A mit der steigenden Flanke von CLK geschrieben. **Ergänze** fehlende Blöcke im Multicycle RISC-V Datenpfad in Abb. 3. Steuere deine neuen Blöcke mit zusätzlichen Kontrollsignalen und adaptiere bzw. ergänze States in der FSM in Abb. 1, um diese Steuersignale zu setzen und so die neue Register File zu unterstützen. **Optimiere** die FSM soweit möglich, damit jede Instruktion so wenig Zyklen wie möglich benötigt.

¹sonic-rv.ics.jku.at

ImmSrc _{1:0}	ImmExt	Instr. Type
00	20×instr[31],instr[31:20]	I-Type
01	20×instr[31],instr[31:25],instr[11:7]	S-Type
10	19×instr[31],instr[31],instr[7],instr[30:25],instr[11:8],b0	B-Type

Tabelle 1: ImmSrc Encoding Tabelle 20×instr[31] ist die Sign Extension auf 32 Bits

Aufgabe 3 – Multicycle RISC-V Analyse

(4+3+3 Punkte)

Betrachte den Multicycle RISC-V Prozessor aus der Vorlesung.

- a) Bestimme die **Länge des kritischen Pfades im Datenpfad** des Prozessors und zeichne ihn in Abb. 2 ein. Analysiere dazu die Länge aller sinnvollen, kritischen Pfade zwischen zwei getakteten Komponenten (z.B. zwischen Registern). (*Tipp: Betrachte nur Pfade, welche auch auftreten können! Zum Beispiel wird das ALU Resultat nie im selben Zyklus in die Memory geschrieben.*)

Entnimm die Verzögerungen der Komponenten aus Tab. 2. Die Verzögerung für nicht gelistete Komponenten soll mit 0 ps angenommen werden. Für die ALU soll in dieser aufgabe ALU_a verwendet werden.

- b) Die ALU_a ist für kurze kombinatorische Laufzeit optimiert, jedoch sehr groß. Deswegen wird ebenfalls eine alternative ALU untersucht, welche weniger Gatter benötigt und dafür langsamer arbeitet. In diesem Fall hätte die neue ALU_b einen Delay von 300 ps. Würde sich der kritische Pfad dadurch ändern? Warum bzw. warum nicht?
- c) Vergleiche die maximale Taktfrequenz des Multicycle Prozessors mit der maximalen Taktfrequenz des **Single-Cycle Prozessor** aus der Vorlesung in Abb. 4. Dazu muss auch für diesen Prozessor der kritische Pfad ermittelt werden. Nimm an, dass für beide Prozessoren ALU_a verwendet wird. Welcher Prozessor hat die **höhere Taktfrequenz**? Welcher Prozessor ist **schneller**?

Tabelle 2: Verzögerungen der Prozessorkomponenten

Komponente	Parameter	Delay [ps]
Register: Clock-to-Q	t_{pcq}	40
Register: Setup	t_{setup}	50
Multiplexer	t_{mux}	35
ALU _a	t_{ALU}	220
Addierer	t_{adder}	150
Extend Einheit	t_{ext}	45
Memory: Lesen	$t_{\text{mem-read}}$	250
Memory: Setup	$t_{\text{mem-setup}}$	50
Register File: Lesen	$t_{\text{RF-read}}$	120
Register File: Setup	$t_{\text{RF-setup}}$	40
ALU _b	t_{ALU}	300

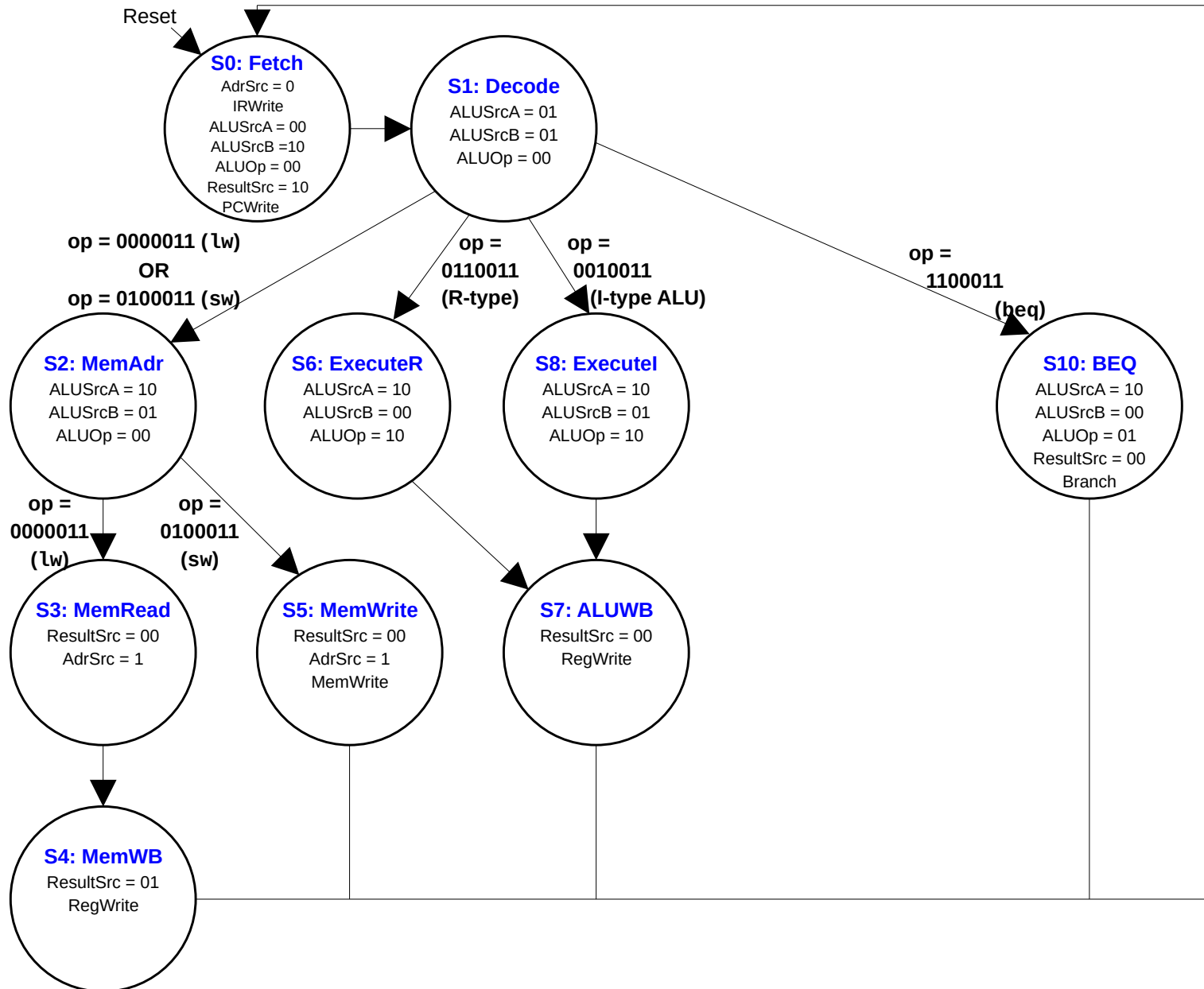


Abbildung 1: Endlicher Automat des Multicycle RISC-V

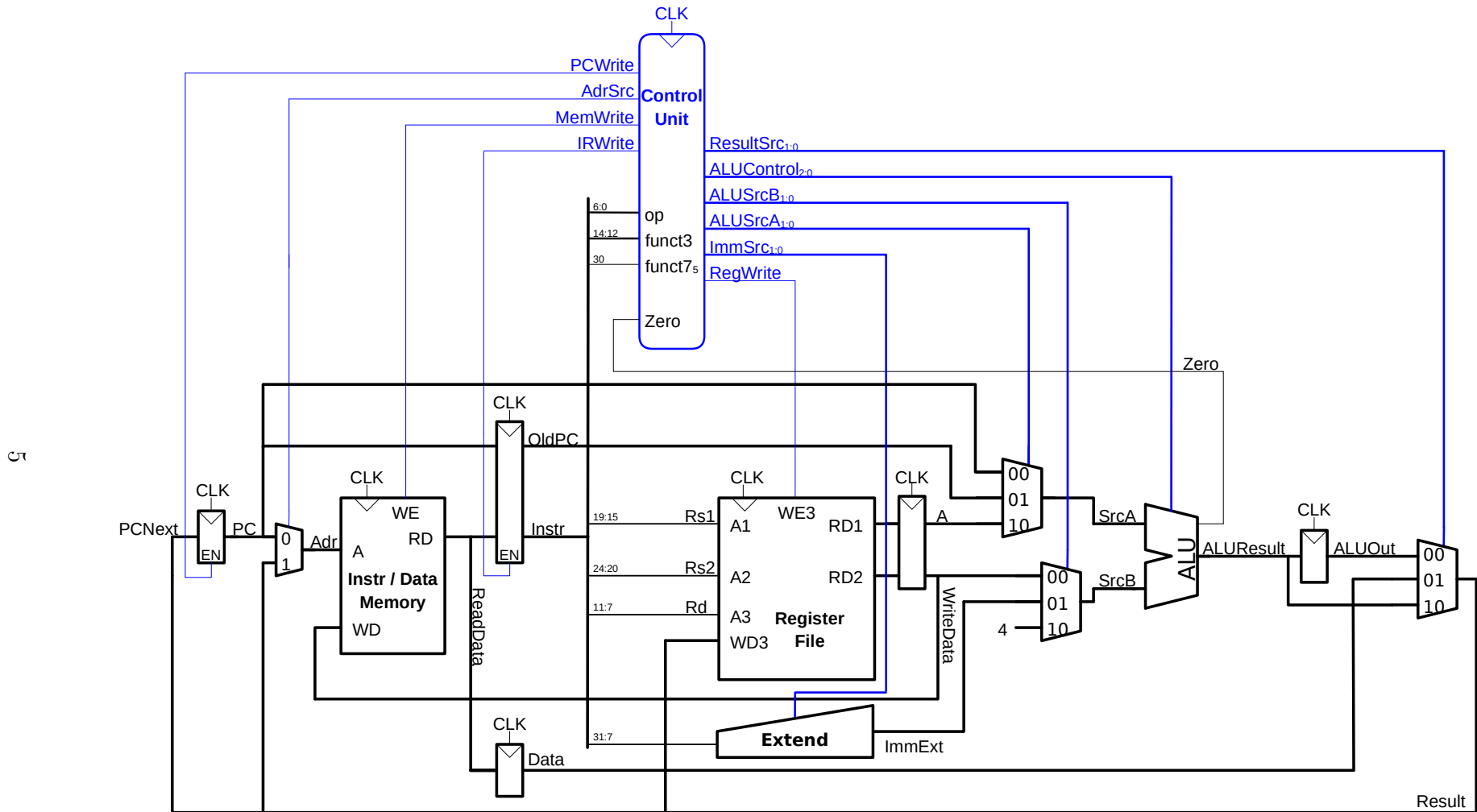


Abbildung 2: Multicycle RISC-V

6

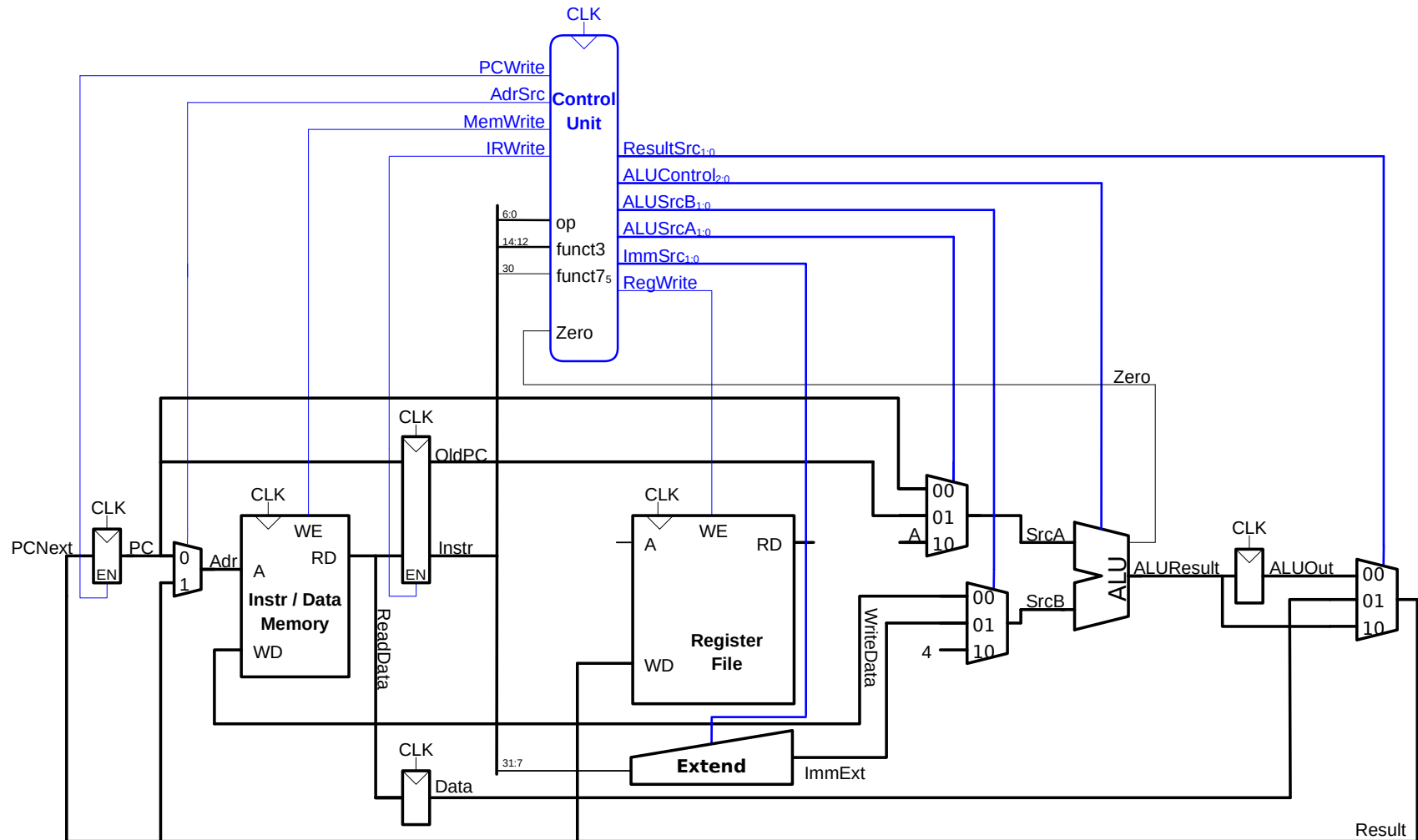


Abbildung 3: Multicycle RISC-V mit 1-Datenport Register File

7

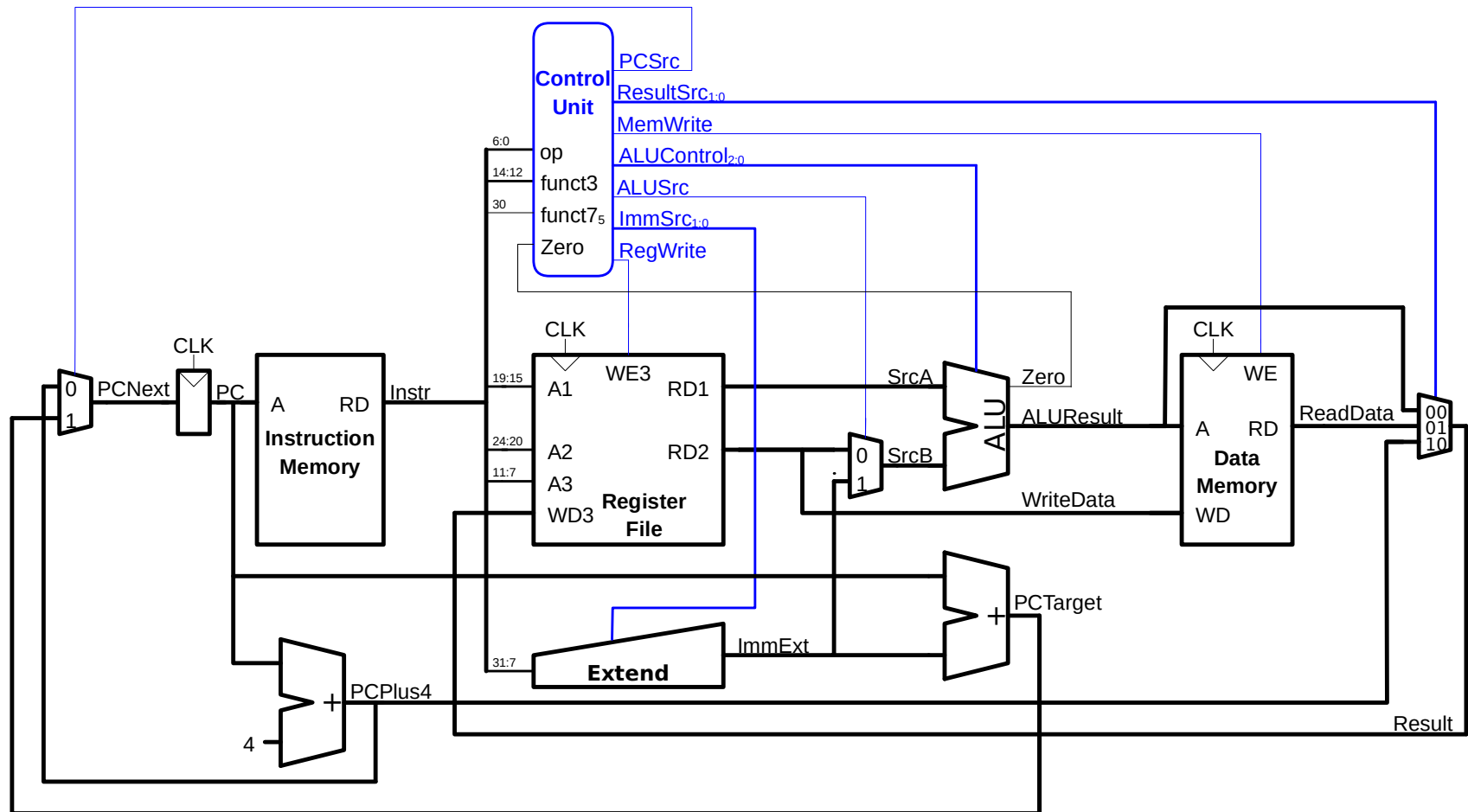


Abbildung 4: Single-Cycle RISC-V